Understanding Performance Trade-offs in Quorum-Based Sequential Consistency

Pietro Farina
Universita' degli Studi di Trento
Trento, Italia
pietro.farina@studenti.unitn.it

***Abstract*—Quorum-based replication is widely used in distributed storage systems to balance consistency and performance. Although quorum intersection rules guarantee sequential consistency, they do not indicate how replication parameters should be selected under realistic workload and failure conditions. This paper presents a systematic performance evaluation of a quorum-based sequentially consistent key–value store with configurable replication, quorum sizes, timeout thresholds, workload skew, and crash injection. Our results quantify how quorum sizes and workload characteristics impact system performance, providing empirical evidence for practical consistency-latency trade-offs. These findings offer both theoretical and operational insights for the design of quorum-based distributed systems.**

***Index Terms*—Distributed systems, quorum-based replication, sequential consistency, performance evaluation, fault tolerance.**

I. Introduction

Distributed storage systems rely on replication to tolerate failures and maintain availability under adverse conditions. A widely adopted approach to replication is quorum-based coordination, where read and write operations must collect acknowledgments from subsets of replicas before completing. By appropriately selecting quorum sizes, systems can guarantee correctness properties while navigating trade-offs between latency, throughput, and fault tolerance.

Quorum-based techniques are grounded in classical voting schemes, which establish that consistency can be preserved when read and write quorums intersect. While these theoretical conditions define correctness boundaries, they do not determine which quorum configuration is preferable under realistic workload conditions. In practice, systems such as Dynamo-inspired architectures and modern distributed databases expose tunable consistency parameters, leaving system designers with the challenge of selecting appropriate configurations based on workload characteristics and failure expectations.

This paper presents a systematic performance evaluation of a quorum-based, sequentially consistent distributed key–value store implemented using the Akka actor framework. The system operates over a ring-based partitioning scheme with configurable replication factor N, read quorum R, write quorum W, and timeout T. Sequential consistency is achieved through versioned data items, quorum intersection rules, and per-key replica locking.

To enable controlled experimentation, we extend the storage system with an autonomous workload generation layer. Clients generate operations according to configurable stochastic models, including exponential inter-arrival times, Bernoulli read/write selection, and Zipf-distributed key access. An Experiment Coordinator enforces quiescent maintenance phases to safely inject membership events and failures. A structured logging subsystem captures both fine-grained protocol events and end-to-end performance metrics, supporting statistical analysis through independent replications and confidence interval estimation.

We conduct a comprehensive evaluation across multiple dimensions: (i) quorum configuration trade-offs under varying read/write mixes, (ii) the impact of network delay and timeout parameters on tail latency and failure rates, (iii) contention effects induced by workload skew and client scaling, (iv) structural recoverability under different crash placement patterns, and (v) the cost of join and leave reconfiguration as replication increases.

Our results show that quorum asymmetry significantly benefits the dominant operation type, network delay primarily affects tail behavior, workload skew induces contention-limited regimes, contiguous failures structurally compromise recoverability, and higher replication factors increase control-plane overhead during membership changes. These findings provide practical insights into the performance implications of quorum parameter selection in sequentially consistent systems.

II. Related Work

The theoretical foundations of quorum-based replication originate from voting-based protocols introduced by Gifford [1], where weighted quorums ensure consistency through intersection properties. The correctness conditions $R+W > N$ and $W > N/2$ guarantee read–write and write–write overlap, preventing stale reads and conflicting updates.

Formal consistency definitions were later established in the distributed computing literature. Sequential consistency, introduced by Lamport [2], requires that operations appear as if executed in a single global order consistent with each client's program order. Stronger models such as linearizability further constrain real-time ordering. While these works rigorously define correctness guarantees, they provide limited guidance on performance implications under stochastic workloads and realistic failure scenarios.

Modern large-scale storage systems adopt quorum-based coordination with tunable consistency parameters. Amazon

Dynamo introduced configurable N, R, and W parameters to balance latency and availability in eventually consistent environments. Apache Cassandra and related systems extend this approach by offering consistency levels such as ONE, QUORUM, and ALL.

These systems demonstrate the practical relevance of quorum trade-offs, yet they primarily target eventual consistency and large-scale deployments. Controlled experimental analyses focusing on sequential consistency, explicit timeout behavior, workload skew, and structural crash patterns remain less extensively documented. Our work complements these systems by providing a systematic simulation-based evaluation under tightly controlled and statistically reproducible conditions.

Beyond system design, prior research has also addressed how realistic workloads should be modeled for performance evaluation. Workload generation in distributed storage systems commonly follows stochastic modeling principles. Exponential inter-arrival times are widely used in performance evaluation due to their memoryless property and analytical tractability, as discussed in performance modeling literature. Operation mixes controlled through probabilistic selection reflect established benchmarking methodologies.

The Yahoo! Cloud Serving Benchmark (YCSB) [4] popularized configurable read/write ratios and Zipf-distributed key selection to emulate realistic skewed access patterns, where a small subset of keys dominates system load. Such skew significantly impacts contention and tail latency behavior. Our workload model adopts these established principles while integrating them into a quorum-based sequential consistency framework with explicit failure and membership modeling.

III. Distributed Storage System

The system under test is a peer-to-peer distributed key-value store, designed to operate without any central entity even in the presence of network delays and node crashes. It provides basic data operations, read and write, and membership operations to model dynamic behavior and faults.

The key requirement of the system is the guarantee of **sequential consistency**: operations issued by clients must appear as if they were executed in a single global order that respects the view of each individual client.

The system is implemented using the Akka actor framework in Java. Actors are used to model storage nodes, clients, and auxiliary management components, allowing the system to support concurrency and failure simulation.

A. Main Actors

Akka actors consist of three components: storage nodes, clients, and a network manager. The **Node** actor represents a storage node in the distributed system. Each node is responsible for maintaining persistent local storage of the data items it is assigned. The **Client** actor represents an information consumer interacting with the storage system, it sends get and update requests to storage node and waits for its response. The **Network Manager** acts as the interface for the distributed storage system and is responsible for initializing the network, creating node and client actors, propagating configuration parameters, and issuing operation to clients and nodes.

B. Data Model

Data keys and node identifiers are mapped to the same identifier space. The key space is treated as a **circular ring**: values increase clockwise and, after the maximum value, wrap around to the minimum. Each node owns a position on this ring given by its identifier, and each data item is placed by locating the first node encountered when moving clockwise from the item's key. Replication is performed by storing the same item in the next $N - 1$ nodes in clockwise order, where N is the replication factor. This yields **predictable placement**, allowing nodes to infer ownership of data replicas by maintaining a local view of the current ring membership.

C. Supported Operation

As anticipated, the system supports two types of operations: data operations, `update(key,value)` and `get(key)`, and membership operations, namely node `join`, `leave`, `crash`, and `recover`.

Data operations are started by clients who contact a node, which will act as the node coordinator for the request. Membership operations are instead started by the Network Manager issuing a request to the target node.

D. Sequential Consistency Guarantees

Quorum-based replication achieves sequential consistency by requiring each operation to involve a sufficiently large subset of replicas. With replication factor N, a write is considered complete after acknowledgments from W replicas, while a read collects responses from R replicas. The particular replicas may change across operations, but correctness depends on the quorum sizes, typically enforced through $R + W > N$ (read–write overlap) and $W > N/2$ (write–write overlap).

Our system targets sequential consistency through (i) fixed, system-wide replication parameters, (ii) versioned data items with quorum-based read/write rules, and (iii) per-key locking to prevent unsafe interleavings.

The parameters N, R, W and the timeout T are fixed at network initialization and never change, ensuring that all nodes apply the same replication and quorum policy. Timeout T bounds the waiting time under network delays or crashes: if an operation reaches T, it is aborted and reported as failed, preventing nodes from blocking indefinitely.

For `get` and `update`, consistency follows from quorum replication and version selection. A `get` returns the value with the highest version among the first R replies, preventing clients from observing stale data. An `update` collects versions, computes a new version strictly greater than those observed, and reports success (including the new version) only after reaching the write quorum W, before propagating the commit to replicas. In addition, replica-side per-key locks serialize accesses locally, avoiding read–write overlaps and write–write conflicts that could expose partial updates or conflicting versions.

Membership changes preserve the same guarantees. During `join` and `recover`, the node must reconcile each key it becomes responsible for by performing an internal read that must reach the normal read quorum R, ensuring it does not introduce stale data into the replication set. During `leave`, the node commits the leave only after receiving acknowledgments from all required destinations, ensuring data is safely handed off before the topology change is finalized.

Note that the choice of read quorum R directly affects recoverability: if set too high, recovery and join operations may become structurally impossible, leading to a permanent loss of availability for some keys. To avoid dead scenarios, R must satisfy $R \leq N - f$, where f is the maximum number of tolerated faults.

E. Model Assumption

The system assumes a FIFO and reliable network: messages are delivered without loss and in the same order they are sent between any pair of actors. Membership and fault-management actions are executed one at a time and only when the system is idle, i.e., when no client operations are in progress. The system also assumes that each node in the system is aware of all other nodes and that at least one node is always alive (not crashed).

IV. Extended Workload Architecture

To enable systematic performance evaluation, the base distributed storage system is extended with a **workload control layer** that orchestrates client activity and membership events without altering the core replication or consistency mechanisms. The storage logic remains unchanged, while the added components regulate how it is exercised during experiments.

Clients operate autonomously, generating operations without external intervention, while an ExperimentCoordinator monitors their idle state to safely trigger membership events. This separation ensures that topology changes are introduced in a controlled manner with respect to the assumptions of the base model.

In addition, a structured logging subsystem records operation phases, latency measurements, and execution outcomes, enabling detailed post-execution analysis.

Together, these mechanisms transform the system into a controllable experimental environment to evaluate the performance impact of quorum-based sequential consistency.

A. Autonomous Clients and Experiment Coordinator

To emulate realistic operating conditions, the system architecture was extended to decouple data operations from manual triggers. In the enhanced model, clients operate as autonomous actors: each client independently generates read and write operations according to configurable probability distributions. This design enables sustained load conditions and supports the evaluation of performance metrics under continuous, statistically controlled traffic.

While autonomous clients provide a realistic workload model, they introduce a new challenge for membership operations. The underlying replication protocol assumes that topology changes occur under quiescent conditions, i.e., when no client operations are in flight. Under continuous autonomous activity, however, the system is rarely idle, making it necessary to explicitly enforce this assumption during experimentation.

To bridge this gap between automated workload generation and the base model requirements, the ExperimentCoordinator component was introduced. This actor serves two primary functions:

- Scheduled Maintenance Phase: It injects membership events based on a configured timeline, removing the need for manual intervention.
- Client State Synchronization: It enforces quiescence by tracking client activity, ensuring that membership operations occur only when no client requests are in progress.

The ExperimentCoordinator periodically initiates a **maintenance phase**. During this phase:

1) A pause message is broadcast to all clients, instructing them to stop generating new operations.
2) Each client completes any ongoing request and replies with an idle acknowledgment.
3) The coordinator waits until acknowledgments have been received from all clients, thereby establishing a globally idle state.
4) Once the idle condition is verified, the coordinator automatically triggers the selected membership operation.
5) Once the membership procedure completes, a resume signal is propagated, allowing clients to restart autonomous execution.

The selection of the membership operation can be either randomized, using configurable probability distributions similarly to data operations, or predefined to simulate specific scenarios. This flexibility allows controlled experimentation under both stochastic and deterministic topology evolution patterns.

B. Workload Randomization

While the ExperimentCoordinator regulates when operations and topology changes may occur, the statistical properties of the workload are defined independently through a parameterized stochastic model. This model governs the generation of client operations, key selection, choice of node, event timing, and communication delays, allowing exploration of different workload conditions.

1) Sources of Randomness: Workload evolution is driven by explicit random variables controlling multiple dimensions of system activity:

- **Operation inter-arrival time**: After completing an operation, each client samples a delay before initiating the next request. This results in a closed-loop workload model, where each client generates a new operation only after the previous one has completed. The inter-arrival delay is modeled using an **exponential distribution**, making the arrival process memoryless with a constant

average rate. This choice follows the standard practice in performance modeling of distributed and web systems, where exponential inter-arrival times lead to Poisson arrival processes. Models built on these assumptions, as discussed by Menascé, Almeida, and Dowdy [3], are widely used for performance evaluation as they balance realism for independent user behavior with analytical simplicity.

- **Operation type selection**: The choice between read and write operations is determined probabilistically, enabling controlled evaluation of read-heavy versus write-heavy workloads. For each operation, the client selects whether to perform a read (`get`) or write (`update`) according to a Bernoulli distribution with configurable probability. This modeling approach follows standard workload-generation practices adopted in database benchmarking frameworks such as YCSB [4], where operation mixes are defined through configurable read and update proportions. By adjusting this probability, experiments can explore read-dominated, write-heavy, or balanced workloads, isolating the performance impact of quorum parameters under different operational mixes.
- **Key Selection**: Keys are selected according to a configurable distribution over the key space. Both uniform and skewed distributions are supported, allowing experiments to capture either evenly distributed access patterns or hot-spot contention scenarios. In particular, the Zipf distribution is widely used in database benchmarking [4] to model realistic access skew, where a small subset of keys receives the majority of requests. Formally, the probability of selecting the k-th ranked key is given by

$$P(k) = \frac{k^{-\theta}}{\sum_{i=1}^{K} i^{-\theta}},$$

  where θ controls the skew intensity. Skewed access patterns increase contention and stress quorum coordination mechanisms, making them especially useful for evaluating load imbalance and worst-case synchronization behavior.
- **Coordinator Selection Policy**: At the beginning of each operation, a client selects an active node to act as the coordinator. The selection is performed by sampling from the current set of active nodes. In the baseline configuration, nodes are chosen according to a uniform distribution, ensuring that each active node has equal probability of being contacted. This simple policy avoids systematic bias toward specific replicas and provides a load-balanced entry point into the system, allowing the evaluation to focus on quorum behavior rather than routing effects.
- **Network Delay**: Communication delay between nodes is modeled using a shifted exponential distribution to capture the two fundamental components of network latency. The shift parameter represents the fixed propagation and transmission delay, i.e., the minimum unavoidable time required for a message to traverse the physical link. This component models hardware-related latency that is largely deterministic and independent of network load. The exponential component captures the variable queuing delay occurring in router and switch buffers, typically characterized by a stochastic distribution [5]. This approach captures communication variability while maintaining analytical simplicity, and it enables evaluation of how latency fluctuations affect quorum coordination, end-to-end operation latency, and timeout behavior. In implementation, sampled delays are bounded to the interval $[\mathit{shift}, d_{\max}]$ to keep simulation time finite and consistent with network reliability assumption.

2) Seed Handling and Reproducibility: All stochastic decisions are generated through a deterministic pseudo-random setup. At experiment initialization, a global run seed is assigned. Each actor (e.g., client, node or coordinator) derives its own deterministic base seed by combining the run seed with its stable actor identifier (the actor path name) through a 64-bit mixing function. This step prevents accidental correlations across actors while ensuring that the same run seed and actor identity always produce the same random sequence.

Within each actor, randomness is further organized into independent streams using SplittableRandom. A root generator is created from the actor-specific base seed and is split exactly once at construction time to obtain one dedicated stream per distribution family (uniform, weighted choice, Bernoulli, Poisson, Zipf, exponential). The split order is fixed and must not change, so that the mapping between distributions and random streams remains stable across versions.

Although deterministic seeding guarantees reproducibility of the random streams, full execution-level reproducibility is not ensured. The system operates in a multi-threaded environment, where thread scheduling and message interleavings are inherently nondeterministic. The primary objective of this design is therefore to guarantee isolation between independent sources of randomness, rather than strict deterministic replay of system execution.

C. Logging Service

To enable systematic performance evaluation, the system incorporates a structured logging subsystem that records both fine-grained execution traces and end-to-end operation metrics. The logging service acts as an instrumentation layer, collecting data for post-execution analysis without influencing the behavior of the distributed protocol.

Logging is organized at two levels of granularity:

- **Operation Summaries**: Each data or membership operation generates a summary record capturing its overall execution characteristics. Summary entries include start and end timestamps, request identifier, coordinator node, operation type and identifier, outcome data and end-to=end latency. These records provide the primary dataset for performance evaluation, enabling computation of metrics such as average and percentile latency, success rate and throughput under varying workload conditions.

- **Execution Events**: In addition to summaries, the system logs intermediate execution events generated by nodes during the processing of an operation. Each event captures timestamp, node and request identifier, operation type, key and version involved, execution phase, phase-level latency and outcome.
  Event-level logging enables fine-grained reconstruction of protocol behavior, including quorum formation timing, replica responsiveness, and timeout occurrences. This level of detail supports deeper latency decomposition and identification of performance bottlenecks.

Each experiment run also records the relevant configuration parameters (e.g., quorum sizes, timeout thresholds, distribution parameters, replication factor). This ensures that every dataset can be unambiguously associated with the corresponding experimental conditions, supporting reproducibility and comparative analysis.

V. Simulation Methodology

This section describes how experiments are conducted and how performance results are obtained. It focuses on the evaluation process and the configuration required.

A. Simulation Regime

The evaluation is performed using a stochastic, event-driven simulation model implemented on top of the Akka actor framework. System state evolves through discrete events such as message deliveries, request completions, timeout expirations, crash injections, and membership changes. Time progression is based on real execution time, with events scheduled via asynchronous messages and timers.

Experiments are executed as long-running runs of this event-driven model under sustained load generated by autonomous clients. The workload follows a closed-loop model where each client issues a new operation only after the previous one has completed.

The evaluation is structured in two phases.

First, **steady-state experiments** are conducted to characterize the system behavior under stable operating conditions. An initial warm-up phase allows the system to stabilize before metrics are collected. Performance measurements are recorded only after this transient startup period, so that reported results reflect stationary behavior.

Second, **finite-horizon experiments** are performed to evaluate system resilience under dynamic conditions. In these experiments, crash and membership events are injected after the system has reached steady-state. The resulting transient response is analyzed in terms of latency degradation, throughput variation and failure rate.

To quantify the uncertainty of our measures, we employed the **independent replication** method. Each experimental configuration is executed multiple times using different random seeds, and each run is treated as an independent replication. This choice allows us to use a single statistical framework for both steady-state and finite-horizon experiments, ensuring methodological consistency across all phases of the evaluation. Empirical inspection confirms consistent convergence behavior across independent runs, indicating that initialization effects do not persist into the measurement phase.

To improve comparability across configurations, we adopt the Common Random Numbers (CRN) technique. Corresponding replications of different configurations are executed using the same initial random seed, ensuring that stochastic workload inputs (e.g., inter-arrival times, operation types, and key selections) are generated from identical random streams. Although execution-level behavior may still diverge due to nondeterministic scheduling, this approach reduces variability caused by workload randomness and enables more controlled comparisons across configurations.

B. Experimental Parameters

Experiments explore different system and workload configurations. The main parameters varied in the evaluation are:

- Replication factor N,
- Read quorum size R,
- Write quorum size W,
- Timeout value T,
- Read probability p, with write probability equal to $1-p$,
- Zipf skew parameter θ,
- Number of clients R,
- Mean network delay $1/\lambda$.

Unless otherwise specified, all other parameters remain fixed to isolate the effect of the variable under study. For readability, detailed configuration tables are provided in the appendix in Table II.

C. Performance Metrics

The following performance metrics are considered.

1) Latency: Operation latency is measured at the client or node level as the time between request submission and completion, depending on data or membership operations. For each run, we compute:

- Median latency (p50)
- 95th percentile latency (p95)
- 99th percentile latency (p99)

These values capture both typical performance and tail behavior.

The reported results correspond to the median across replications. 95% Confidence intervals are computed using bootstrap resampling over the set of independent runs.

2) Throughput: Throughput is defined as the number of successfully completed operations per second during the steady-state phase.

Throughput is computed separately for each run and then averaged across replications. Corresponding 95% confidence intervals are computed using the Student-t distribution over the independent replications.

3) Failure Rate: The failure rate is defined as the fraction of operations that do not complete successfully. This metric quantifies the reliability of a given configuration under the specified workload.

Failures are further classified according to their cause, allowing the failure rate to be decomposed into:

- Timeout rate
- Busy or unavailability rate (e.g., replica not available or request rejected due to ongoing operations)

It can be further splitted into Timeout Rate and Busy Rate based on the reason of For each run, failure rates are computed over all completed operations. Final reported values correspond to the mean across independent replications, and 95% confidence intervals are computed using the Student-t distribution.

VI. Results

This section presents the experimental evaluation of the system under different workload and failure conditions. We first analyze quorum configuration trade-offs under varying read/write mixes. We then study the impact of network delay and timeout parameters, followed by contention effects induced by workload skew and client scaling. Finally, we examine system resilience under crash scenarios and the cost of membership reconfiguration.

A. Impact of Quorum Size on Performance

The first objective of this experiment is to analyze how quorum configuration impacts system performance under different workload compositions. In quorum-based replication, the choice of R and W directly influences latency, throughput, and consistency guarantees. The optimal configuration therefore depends on the relative dominance of read and write operations.

In read-dominated workloads, minimizing the read quorum size reduces the coordination overhead required to complete the most frequent operation. This is expected to lower average and tail read latency, which in turn improves overall throughput. Conversely, write-dominated workloads benefit from smaller write quorums, as write latency becomes the dominant contributor to system delay.

The experimental results confirm this intuition. Configurations with smaller quorum size for the dominant operation exhibit reduced tail latency (Figure 1) and increased throughput in Figure 2. This behavior is consistent with the reduced number of replicas involved in the critical path of the most frequent requests, which lowers coordination delay and decreases the probability of slow responses propagating to the tail of the latency distribution.

In particular, under read-dominated workloads, configuration R1W3 achieves the lowest GET p99 latency, while under write-dominated workloads R3W1 minimizes UPDATE p99 latency. The balanced configuration R2W2 provides intermediate performance for both operations, reflecting the symmetric quorum cost.

Interestingly, a read dominance such as 70/30 is not sufficient to make R1W3 strictly preferable from a throughput perspective: although GET latency is minimized, the overall GET throughput remains higher under R2W2 configuration.

Confidence intervals remain relatively tight across replications, indicating stable behavior and limited run-to-run variability. Slightly wider intervals are observed when the dominant operation requires a larger quorum, which is expected due to increased coordination overhead and higher sensitivity to replica response variability. Nevertheless, the separation between configurations is sufficiently pronounced to support the observed trends.

For completeness, we also include the configuration $R = 3, W = 1$ as a reference case. Although this setting improves write latency, it does not satisfy the quorum intersection requirement $W > N/2$, and therefore does not guarantee write-write conflict resolution. As expected, the results show occurrences of overwritten writes, whose frequency is reported in Appendix at Table III.

B. Effect of Network Delay and Timeout

To assess the robustness of the system to communication slowdown, we vary the network delay while maintaining a fixed workload and quorum configuration. Because quorum operations depend on collecting responses from multiple replicas, communication delay directly influences the operation critical path and can interact with timeout mechanisms.

As expected, increasing network delay significantly impacts tail latency. Since each operation requires multiple request–reply exchanges, the delay accumulates leading to an approximately linear growth of p99 latency with respect to the network delay level (Figure 3). The timeout parameter primarily affects the upper bound of the tail, shifting the p99 when the network delay approaches the configured threshold. Throughput exhibits the complementary behavior. Under the closed-loop workload model, clients generate new requests only after completion of the previous one; therefore, increased latency directly reduces the number of completed operations per second.

Notably, the reject/busy rate increases more significantly than the timeout rate as network delay grows (Figure 4). This behavior is explained by the closed-loop workload model and increased service time. As communication delay increases, operations remain in the system for longer, raising node occupancy and leading to higher rejection due to capacity limits. In contrast, timeouts occur only when operation completion time exceeds the configured threshold. Since latency grows gradually and remains below the timeout value for most delay levels, timeout rates increase more slowly and become dominant only when delay approaches the timeout threshold.

C. Impact of Skew and Client Scaling

While previous experiments focused on communication delay and quorum configuration, we want now to analyze contention effects arising from workload distribution and client scaling.

Increasing the number of clients raises the offered load and system occupancy. Under uniform access patterns, throughput scales approximately proportionally with the number of clients

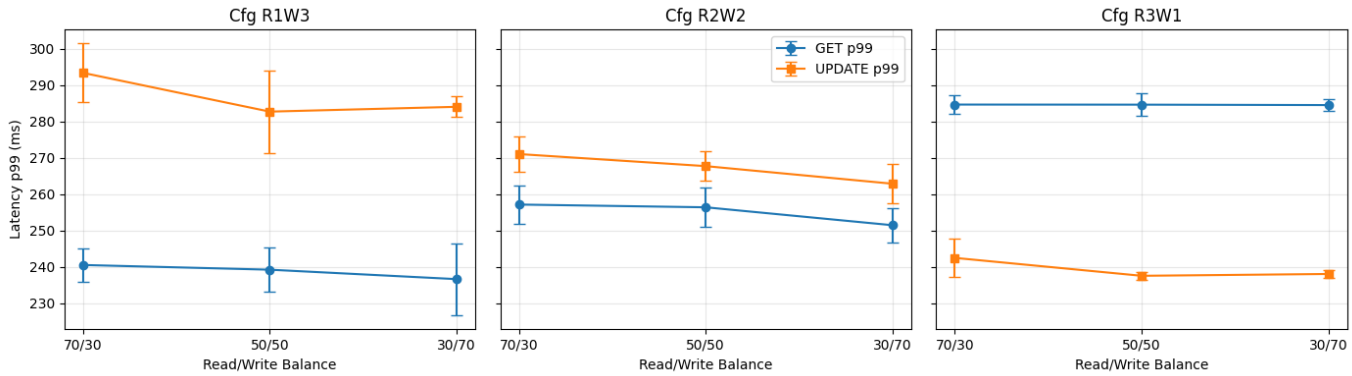

Fig. 1. Tail Latency Comparison Across Varying Read-Write Workloads. Error bars denote 95% confidence intervals.

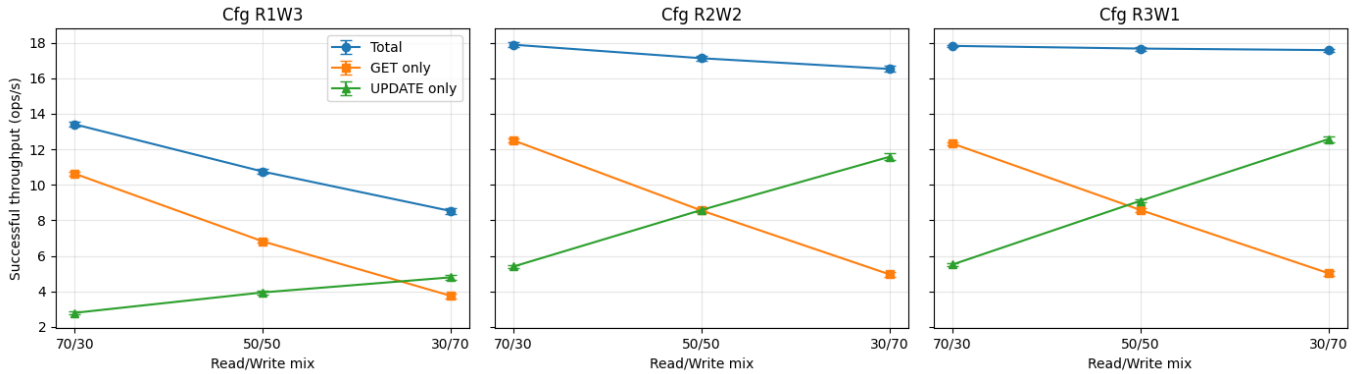

Fig. 2. Throughput Comparison Across Varying Read-Write Workloads. Error bars denote 95% confidence intervals.

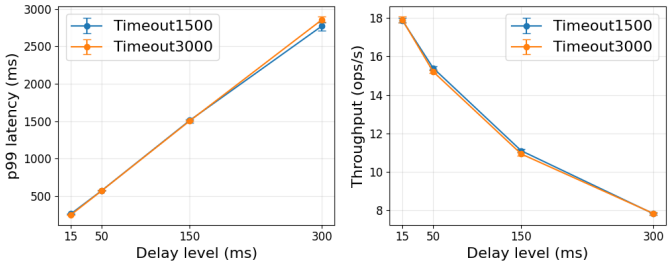

Fig. 3. Tail latency and throughput under different delay level.

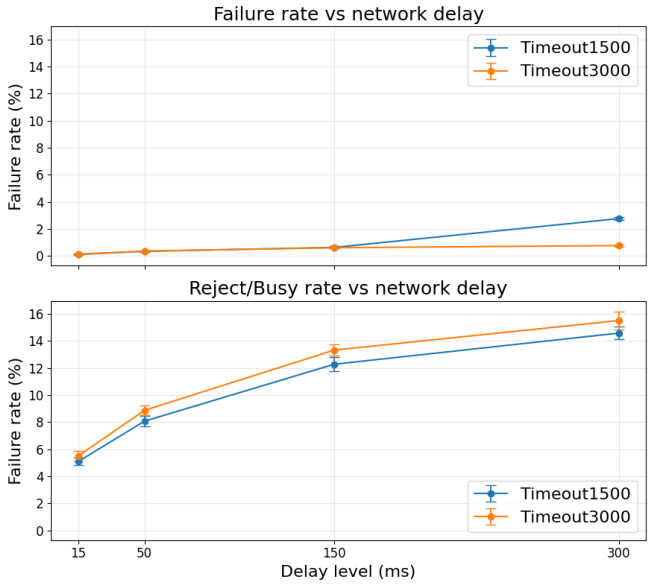

Fig. 4. Failure rates under different delay level with 95% confidence intervals. Busy failures exhibit a significantly steeper increase compared to timeout errors.

until capacity limits are approached. However, when the workload exhibits skew (higher Zipf parameter), a small subset of keys becomes hot, repeatedly targeting the same replicas and increasing coordination conflicts.

The results confirm these expectations. Figure 5 shows throughput increasing with the number of clients in all cases, but degrading significantly as skew intensifies. For example, at 50 clients, throughput under high skew (Zipf $\theta = 1.5$) is substantially lower than under near-uniform access.

This effect is even more evident in the reject/busy rate shown in Figure 6. While uniform workloads exhibit limited rejection even at higher client counts, skewed workloads rapidly enter a contention-limited regime, with reject rates

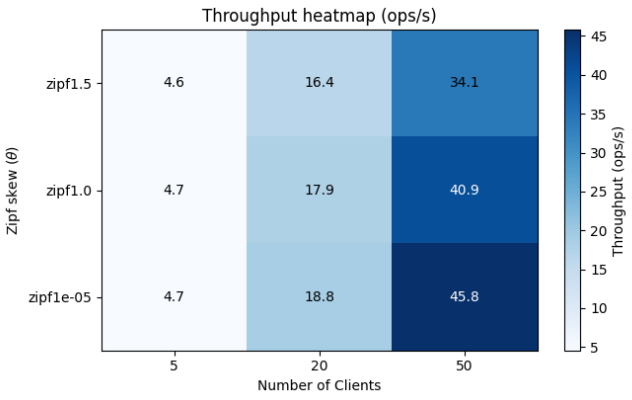

Fig. 5. Throughput across varying client counts and request skews (θ).

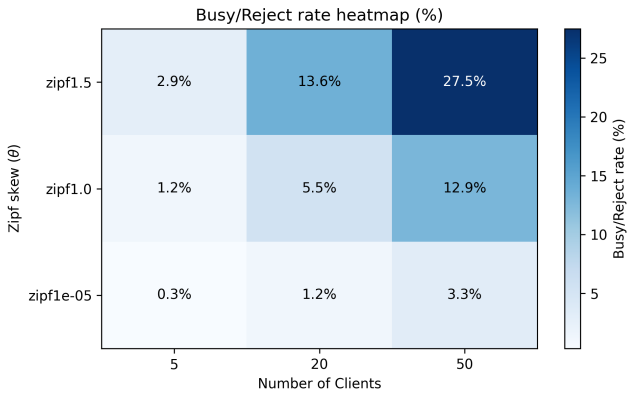

Fig. 6. Rate of operations failing due to replica busy status across varying client counts and request skews (θ).

exceeding 25% at high concurrency. This behavior indicates that performance degradation is primarily driven by hotspot contention rather than communication delay or timeout mechanisms.

It is worth noting that the client scaling explored in this section does not correspond to the absolute capacity limit of the system. Additional stress experiments (reported in appendix Table IV) show that the system can sustain significantly higher aggregate throughput when the number of clients is increased substantially (e.g., hundreds of concurrent clients), albeit with increasing failure rates. Therefore, the degradation observed in the heatmaps should be interpreted as a contention-driven regime induced by workload skew, rather than full network saturation.

D. Resilience Under Failure and Workload Skew

This experiment evaluates the impact of node crashes under both uniform and skewed (Zipf) workloads, and investigates how placement of crashed nodes affects recovery feasibility. We consider two crash patterns: contiguous failures, where adjacent nodes in the ring crash, and non-contiguous failures, where crashed nodes are separated on the ring. Since replicas are placed on successive nodes, contiguous failures remove overlapping replica sets for a subset of keys, causing multiple replicas of the same key to become unavailable simultaneously. In contrast, non-contiguous failures tend to distribute replica losses across different keys, so most keys retain enough surviving replicas to satisfy quorum requirements.

Recovery feasibility is therefore primarily determined by replica overlap. As supported by Figure 7, in the contiguous case some keys lose multiple replicas at once and may no longer satisfy the required R and W quorums, leading to sustained availability loss and pronounced throughput degradation. In the non-contiguous case, enough replicas typically remain to satisfy quorum for most keys, and the system continues operating with reduced capacity. This structural behavior is observed under both uniform and skewed workloads.

Under skewed workloads, hot keys concentrate a large fraction of traffic on a limited subset of replicas. Consequently, one might expect crashes to have a more pronounced impact compared to uniform workloads, as the failure of replicas storing hot keys could significantly disrupt a substantial portion of requests. Interestingly, the results show that aggregate throughput remains comparable to the uniform case, both under contiguous and non-contiguous crash scenarios. This behavior can be explained by the closed-loop workload model. Clients are not blocked indefinitely when an operation fails; instead, after completion (successful or failed), they sample a new key and issue another request. As a result, failed operations increase the observed failure rate but do not prevent clients from contributing further requests to the system. This effect is consistent with the noticeably higher failure rate observed under Zipf workloads compared to uniform ones as shown in Appendix at Figure 8.

E. Membership Reconfiguration Time

Finally, we are interested in evaluating the time required to complete join and leave operations as a function of the replication factor. Although membership changes are executed during quiescent maintenance phases and do not directly affect client-visible latency, they incur control-plane overhead in the form of state transfer and replica reassignment. Quantifying this cost is important to understand the tradeoff between higher replication levels and maintenance complexity.

As the replication factor increases, each key is stored on more replicas, increasing the total replicated state in the system and allowing the system to withstand more failures. On the other hand, join and leave operations are expected to require more coordination and data movement to restore replication invariants. We therefore expect reconfiguration time to grow with the replication factor for both join and leave events.

Table I reports the median time to complete join and leave operations as the replication factor increases. Two main observations emerge. First, join operations are consistently more expensive than leave operations across all configurations, requiring approximately twice the completion time. This asymmetry reflects the additional coordination and state acquisition required to integrate a new node into the replica set, compared to redistributing responsibilities after a departure. Second, join

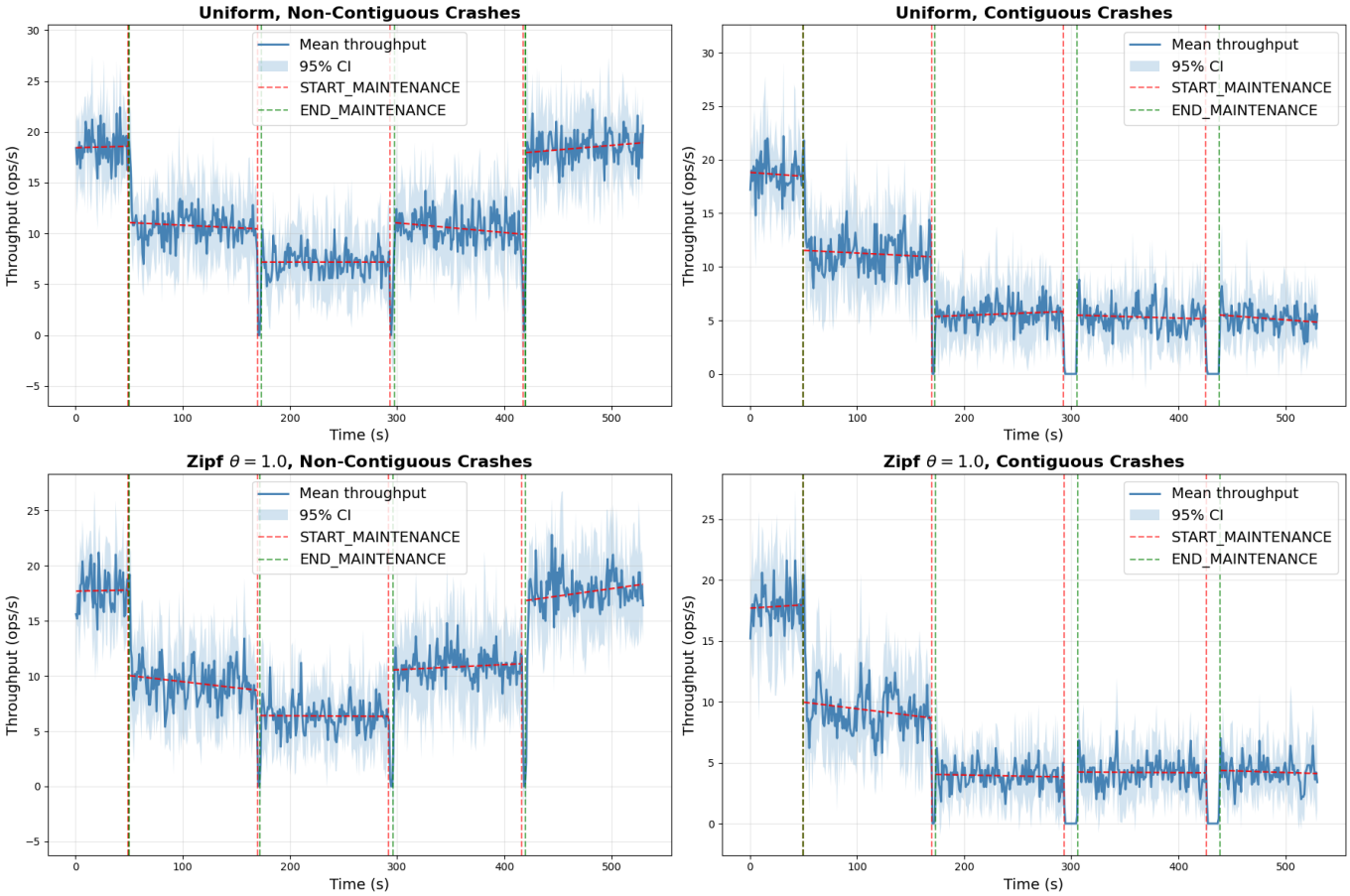

Fig. 7. Impact of failure locality on system throughput. Contiguous crashes lead to localized quorum loss for specific replica sets.

latency increases monotonically with the replication factor, indicating that higher replication levels introduce additional coordination and state-transfer overhead during integration. In contrast, leave latency exhibits only a minor increase as replication grows.

TABLE I
MEMBERSHIP OPERATION LATENCY (MS) (95% CI) ACROSS INCREASING REPLICATION FACTORS (N)

Operation	$N = 3$	$N = 5$	$N = 7$
JOIN	235.50 [215.0, 265.0]	239.50 [214.5, 269.5]	273.00 [236.0, 298.0]
LEAVE	117.00 [100.0, 138.5]	115.00 [109.0, 126.0]	125.00 [115.0, 133.5]

VII. CONCLUSION

This work presented a systematic performance evaluation of a quorum-based, sequentially consistent distributed key–value store under realistic stochastic workloads and dynamic conditions. By combining configurable quorum parameters, workload skew, network delay modeling, crash injection, and membership reconfiguration, we investigated how replication choices affect both steady-state performance and resilience.

Our results confirm that quorum configuration must be aligned with workload composition: reducing the quorum size for the dominant operation type significantly improves tail latency and overall throughput. Network delay primarily impacts tail latency and indirectly reduces throughput under closed-loop load, while timeout parameters shape the upper bound of latency distributions. Workload skew introduces contention-limited regimes where hotspot replicas become bottlenecks, increasing reject rates even before global capacity limits are reached. Crash placement critically influences recoverability: contiguous failures structurally reduce replica overlap and may render quorum satisfaction impossible for subsets of keys. Finally, increasing the replication factor improves fault tolerance but introduces measurable overhead in membership reconfiguration, particularly for join operations.

Overall, our findings highlight that quorum parameters should not be selected statically or purely based on correctness conditions. Instead, they must be tuned according to workload characteristics, expected delay variability, and failure assumptions. Future work could investigate adaptive quorum selection strategies, network partition modeling, stronger consistency guarantees such as linearizability, or validation on real distributed deployments.

REFERENCES

[1] David K. Gifford. 1979. Weighted voting for replicated data. In Proceedings of the seventh ACM symposium on Operating systems principles (SOSP '79). Association for Computing Machinery, New York, NY, USA, 150–162. https://doi.org/10.1145/800215.806583

[2] Lamport, L. (1979). How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs. IEEE Transactions on Computers, C-28(9), 690–691

[3] Menascé, Daniel & Almeida, Virgilio & Dowdy, L.. (2004). Performance by Design: Computer Capacity Planning by Example.

[4] Cooper, Brian & Silberstein, Adam & Tam, Erwin & Ramakrishnan, Raghu & Sears, Russell. (2010). Benchmarking cloud serving systems with YCSB. Proceedings of the 1st ACM Symposium on Cloud Computing, SoCC '10. 143-154. 10.1145/1807128.1807152.

[5] Mukherjee, A. (1992). On the dynamics and significance of low frequency components of Internet load. Internetworking: Research and Experience, 3(4), 163-205.

APPENDIX A
SUPPLEMENTARY EXPERIMENTAL DETAILS

TABLE II
EXPERIMENTAL CONFIGURATION PARAMETERS, QUORUMS

Parameter	Quorum Trade-off	Network Delay	Skew & Scaling	Failures	Membership	Stress Test
(N, R, W)	{(3,3,1), (3,2,2), (3,1,3)}	(3,2,2)	(3,2,2)	(3,2,2)	{(3,2,2), (5,3,3), (7,4,4)}	(3,2,2)
Timeout T (ms)	1500	{1500, 3000}	3000	3000	3000	3000
Zipf skew θ	1.0	1.0	{1.0E-5, 1.0, 1.5}	1.0	1.0E-5	1.0
Read probability p_r	{0.7, 0.5, 0.3}	0.7	0.7	0.7	0.5	0.7
Poisson λ (ops/s)	1.0	1.0	1.0	1.0	1.0	1.0
Exponential λ (ms^{-1})	1/15	{1/15, 1/50, 1/150, 1/300}	1/15	1/15	1/15	1/15
Shift (ms)	5	5	5	5	5	5
Tail cap (ms)	2000	2000	2000	2000	2000	2000
Key space size	50	50	50	50	50	50
Number of nodes	10	10	10	10	10	10
Number of clients	20	20	{5, 20, 50}	20	50	{50, 100, 50, 1000}

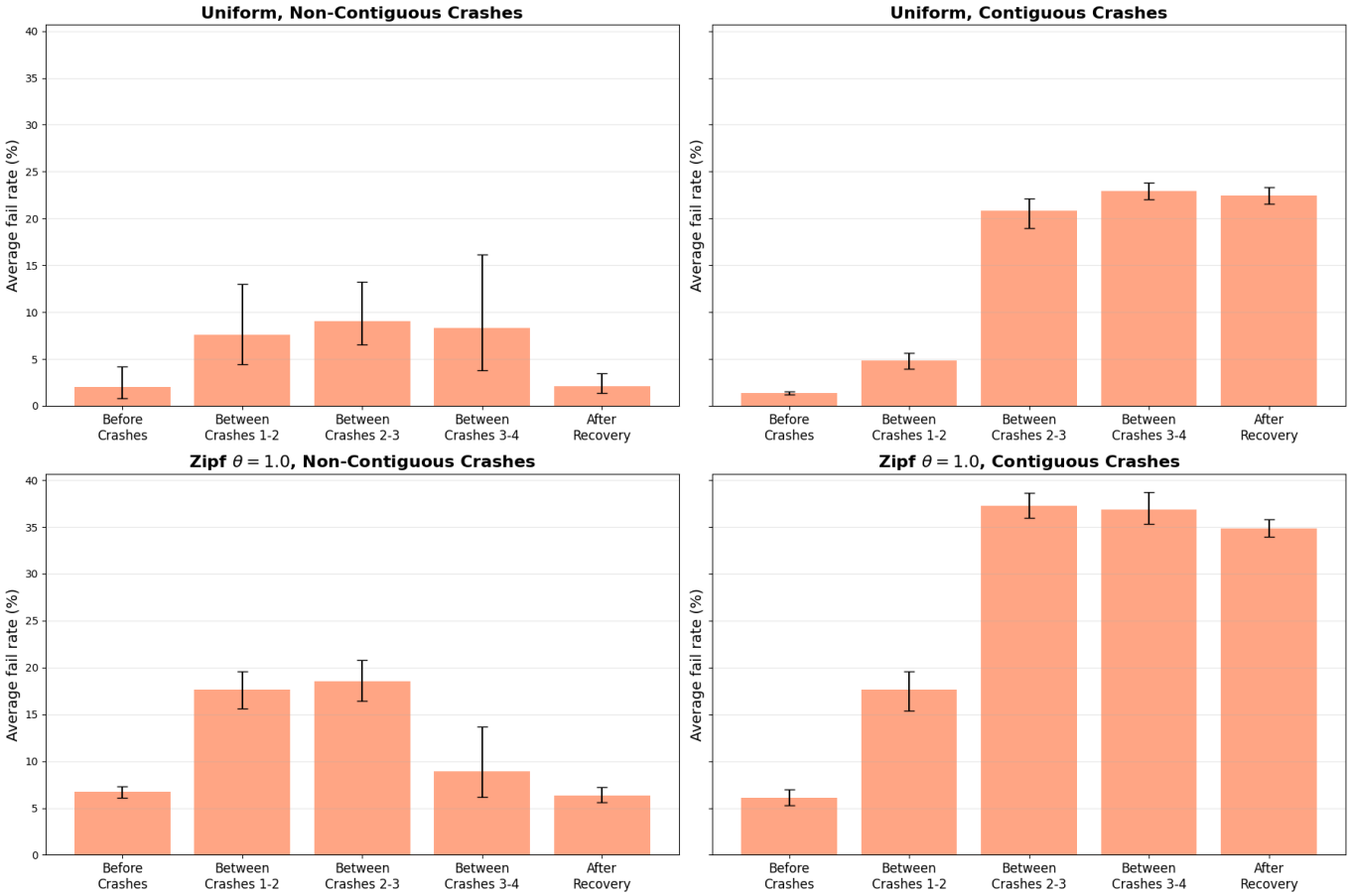

Fig. 8. Quorum failure rates during node crashes. Higher Zipf skew leads to increased unavailability as "hot" replica sets lose their required R/W majority

TABLE III
MISSED WRITE CONFLICTS UNDER $R = 3$, $W = 1$ (95% CI)

Workload	Missed Conflicts (avg)	Conflict Rate (%)
Read-dominant	9.30 [6.56, 12.04]	0.55 [0.39, 0.71]
Balanced	27.40 [23.51, 31.29]	0.98 [0.85, 1.11]
Write-dominant	49.00 [43.65, 54.35]	1.26 [1.12, 1.40]

TABLE IV
STRESS TEST UNDER ZIPF SKEW ($\theta = 1$): IMPACT OF CLIENT SCALING (95% CI)

Clients	Throughput (ops/s)	Busy/Reject Rate (%)
50	45.78 [45.61, 45.95]	3.3 [3.2, 3.4]
100	73.88 [73.23, 74.53]	20.7 [20.3, 21.2]
500	376.12 [332.44, 422.39]	55.1 [51.4, 58.7]
1000	405.79 [404.16, 407.41]	57.6 [57.4, 57.7]